

CO5-AT1

Pseudocode Writing Task (Algorithm Design)

Poojitha — Computer Science, SIMATS Engineering

Part 1: Social Network Graph Analysis

A social network is modeled as an undirected graph $G = (V, E)$, where each vertex represents a user and each edge represents a connection (friendship/follow) between two users.

1.1 Finding Connected Components (BFS-based)

Goal: Identify all separate clusters of users where every user in a cluster can reach every other user through some chain of connections.

```
ALGORITHM FindConnectedComponents(Graph G)
  Input:  $G = (V, E)$  //  $V$  = users,  $E$  = friendships/connections
  Output: List of components, each a set of users

  visited    <- empty set
  componentList <- empty list
  componentId <- 0

  FOR each vertex  $u$  in  $V$ :
    IF  $u$  NOT IN visited:
      componentId <- componentId + 1
      currentComponent <- empty set
      queue <- empty queue
      ENQUEUE(queue,  $u$ )
      ADD  $u$  to visited

      WHILE queue is not empty:
         $v$  <- DEQUEUE(queue)
        ADD  $v$  to currentComponent

        FOR each neighbor  $w$  of  $v$  in  $G$ :
          IF  $w$  NOT IN visited:
            ADD  $w$  to visited
            ENQUEUE(queue,  $w$ )

      ADD currentComponent to componentList

  RETURN componentList
END
```

Complexity: $O(V + E)$ — each vertex and each edge is visited exactly once (BFS traversal).

1.2 Shortest Connection Path Between Two Users (BFS)

Goal: Suggest the shortest chain of mutual connections linking a source user to a target user. Plain BFS finds the shortest path by number of hops (unweighted graph).

```
ALGORITHM ShortestPath(Graph G, source, target)
  Input:  $G = (V, E)$ , source user, target user
```

Output: Shortest path (list of users) or "No connection"

IF source and target are NOT in the same component:
RETURN "No connection"

visited <- empty set
parent <- empty map
queue <- empty queue

ENQUEUE(queue, source)
ADD source to visited
parent[source] <- NULL

WHILE queue is not empty:
current <- DEQUEUE(queue)

IF current == target:
RETURN BuildPath(parent, target)

FOR each neighbor n of current in G:
IF n NOT IN visited:
ADD n to visited
parent[n] <- current
ENQUEUE(queue, n)

RETURN "No connection"

FUNCTION BuildPath(parent, target)
path <- empty list
node <- target
WHILE node != NULL:
INSERT node at front of path
node <- parent[node]
RETURN path
END

Note: If connections carry weights (e.g., interaction strength or cost), replace BFS with Dijkstra's Algorithm using a min-priority queue, since plain BFS guarantees the shortest path only by hop count, not by weighted cost.

Complexity: $O(V + E)$ for BFS; $O((V + E) \log V)$ if Dijkstra's Algorithm is used for weighted connections.

Part 2: Task Scheduling System (Project Management)

Real-world scenario: a project management tool must schedule tasks that depend on one another (e.g., "Design" must finish before "Development" begins). This is modeled as a Directed Acyclic Graph (DAG).

2.1 Modeling the Problem as a Directed Graph

- Each task = a vertex.
- Each dependency "Task A must finish before Task B" = a directed edge $A \rightarrow B$.
- Represented using an adjacency list: $\text{adj}[\text{task}] = \text{list of dependent tasks}$.
- Maintain $\text{inDegree}[\text{task}] = \text{number of prerequisite (incoming) tasks}$.

2.2 Ensuring No Cyclic Dependencies Exist (Cycle Detection)

A cycle among task dependencies would make scheduling impossible (e.g., A depends on B, and B depends on A). Detect cycles using DFS with vertex coloring.

```
ALGORITHM HasCycle(Graph G)
Input: G = (V, E) // directed graph of tasks
Output: TRUE if a cycle exists, else FALSE

color <- map, all vertices set to WHITE
// WHITE = unvisited, GRAY = in progress, BLACK = done

FOR each vertex v in V:
  IF color[v] == WHITE:
    IF DFSVisit(v, color, G) == TRUE:
      RETURN TRUE

RETURN FALSE

FUNCTION DFSVisit(u, color, G)
  color[u] <- GRAY

  FOR each neighbor v of u in G:
    IF color[v] == GRAY:
      RETURN TRUE // back edge found -> cycle
    IF color[v] == WHITE:
      IF DFSVisit(v, color, G) == TRUE:
        RETURN TRUE

  color[u] <- BLACK
  RETURN FALSE
END
```

Complexity: $O(V + E)$ — standard DFS traversal.

2.3 Generating a Valid Execution Order (Topological Sort — Kahn's Algorithm)

Once the graph is confirmed acyclic, a valid task execution order is produced using Kahn's Algorithm (BFS-based topological sort).

```
ALGORITHM TopologicalSort(Graph G)
Input: G = (V, E), a directed acyclic graph (DAG) of tasks
Output: A valid task execution order, or an error if a cycle exists

inDegree <- map, initialized to 0 for all vertices
FOR each edge (u, v) in E:
  inDegree[v] <- inDegree[v] + 1

queue <- empty queue
FOR each vertex v in V:
  IF inDegree[v] == 0:
    ENQUEUE(queue, v) // tasks with no prerequisites

executionOrder <- empty list

WHILE queue is not empty:
  u <- DEQUEUE(queue)
  ADD u to executionOrder

  FOR each neighbor v of u in G:
```

```

    inDegree[v] <- inDegree[v] - 1
    IF inDegree[v] == 0:
        ENQUEUE(queue, v)

    IF LENGTH(executionOrder) != LENGTH(V):
        RETURN "Error: Cycle detected -- no valid execution order"
    ELSE:
        RETURN executionOrder
END

```

Complexity: $O(V + E)$. If executionOrder omits any vertex, a cycle exists — HasCycle() confirms this before scheduling.

Part 3: Minimum Spanning Tree — Kruskal's Algorithm

Scenario: A city planning department must connect multiple zones using underground fiber-optic cables at minimum total installation cost, with no redundant (cyclic) connections.

3.1 Problem Mapping

- Vertex = a city zone.
- Edge (u, v, weight) = a possible cable connection with its installation cost.
- Goal: connect all zones at minimum total cost with no cycles — a classic Minimum Spanning Tree (MST) problem.

3.2 Kruskal's Algorithm — Pseudocode

```

ALGORITHM KruskalMST(Graph G)
    Input: G = (V, E), where each edge has a weight (installation cost)
    Output: MST -- the set of edges connecting all zones at minimum cost

    // Step 1: Sort all edges in non-decreasing order of weight
    SORT E by weight ascending

    // Step 2: Initialize a Disjoint Set (Union-Find), one set per zone
    FOR each vertex v in V:
        MAKE-SET(v)

    mstEdges <- empty list
    totalCost <- 0

    // Step 3: Process edges in sorted order
    FOR each edge (u, v, weight) in sorted E:
        IF FIND(u) != FIND(v):    // adding this edge will not form a cycle
            ADD (u, v, weight) to mstEdges
            totalCost <- totalCost + weight
            UNION(u, v)

    IF LENGTH(mstEdges) == LENGTH(V) - 1:
        BREAK    // MST complete (V-1 edges connect V vertices)

    IF LENGTH(mstEdges) != LENGTH(V) - 1:
        RETURN "Graph is disconnected -- cannot fully connect all zones"

    RETURN mstEdges, totalCost

```

```

// ---- Disjoint Set (Union-Find) helper operations ----

FUNCTION MAKE-SET(x)
  parent[x] <- x
  rank[x] <- 0

FUNCTION FIND(x)
  IF parent[x] != x:
    parent[x] <- FIND(parent[x]) // path compression
  RETURN parent[x]

FUNCTION UNION(x, y)
  rootX <- FIND(x)
  rootY <- FIND(y)
  IF rootX == rootY:
    RETURN // already connected, would form a cycle

  // union by rank
  IF rank[rootX] < rank[rootY]:
    parent[rootX] <- rootY
  ELSE IF rank[rootX] > rank[rootY]:
    parent[rootY] <- rootX
  ELSE:
    parent[rootY] <- rootX
    rank[rootX] <- rank[rootX] + 1
END

```

Complexity: $O(E \log E)$ for sorting the edges, plus $O(E \alpha(V))$ for Union-Find operations (α = inverse Ackermann, effectively constant) $\rightarrow$ overall $O(E \log E)$.

3.3 How This Satisfies the City Planning Constraints

Constraint	How Kruskal's Algorithm Satisfies It
All zones must be connected	The algorithm halts only after adding $V - 1$ edges, which connects all V zones (provided the input graph is connected).
Total installation cost must be minimized	Edges are always considered in ascending order of weight; this greedy strategy is proven optimal for constructing an MST.
No redundant connections (no cycles)	The check $\text{FIND}(u) \neq \text{FIND}(v)$ rejects any edge that would join two zones already in the same connected set, preventing cycles.
Network must remain fully connected	A spanning tree is by definition connected, using exactly $V - 1$ edges — the minimum number of edges required for full connectivity.